Aspis: Transparent Private-Spend Verification on Solana with Lean Formalization and Rust-to-Lean Correspondence

Dominic Barker
Independent Researcher
DJBarker87/aspis

Draft 0.1 – 25 July 2026

Draft status. This manuscript is a complete first publication draft, but it is not yet the immutable submission version. The V5 mainnet signature supplied for this draft is recorded in section 9.4; the repository's machine-readable release record had not yet imported the landed slot, compute consumption, deployed ProgramData identity, proof digest, and post-state evidence at the manuscript cut-off. Those fields are deliberately marked open rather than inferred.

Abstract

We present *Aspis*, a transparent, hash-based argument and Solana program for a narrow private-spend state transition. A prover shows that a committed input note belongs to the current pool, is authorized by a secret owner key, has not previously been spent, and is replaced by an output note satisfying the asset and integer-value rules. The final Solana instruction verifies the proof, advances the pool root, and records a canonical one-time spent marker. These effects are atomic: either all checks and writes succeed or no persistent state changes. The construction uses no trusted setup ceremony.

Aspis is designed as a traceable chain of evidence rather than only as an on-chain benchmark. A Lean 4 development formalizes substantial parts of the spend relation, concrete release-security calculations, circle-group and hiding arguments, pinned Poseidon2 executions, and the maintained models of the current verifier. Selected production Rust is extracted with Charon and translated to Lean with Aeneas; additional Lean bridge proofs connect the generated definitions to the mathematical models for the stated V5 release path. The source-to-model result covers the selected Component-A schedule, Components B and C, the public output, Tag-67 instruction decoding, and six ordered proof-of-work checks, subject to one explicit equation at the transcript-hash function-call boundary. The exact V5 Solana program is separately reproducible byte-for-byte from a pinned source commit and build environment.

The earlier q18/g37 release finalized on Solana mainnet with a 65,407-byte proof and consumed 1,344,003 of the 1,400,000 compute-unit transaction limit. The current V5 binary is 1,258,496 bytes, finalized its full state transition on devnet at 1,335,952 compute units, and has a release-policy ceiling of 1,356,912 compute units under an Agave 4.1.0 replay. A V5 mainnet transaction signature is included in this draft; its complete landed evidence will be inserted from the final release record rather than reconstructed from an explorer interface.

The demonstrated protocol is deliberately limited: one input, one output, a same-path tree replacement, one sequential pool, no deposit or append path, and no growing anonymity set. We separate argument soundness from the additional knowledge premise needed for theft resistance, state the formal and trusted boundaries explicitly, and publish the source, proof artefacts, formal developments, build records, runtime measurements, and release checks required for independent review.

1 Introduction

Public blockchains make state transitions easy to audit and difficult to conceal. A payment protocol that hides note ownership or value must therefore prove, in public, that a hidden transition obeys the ledger rules.

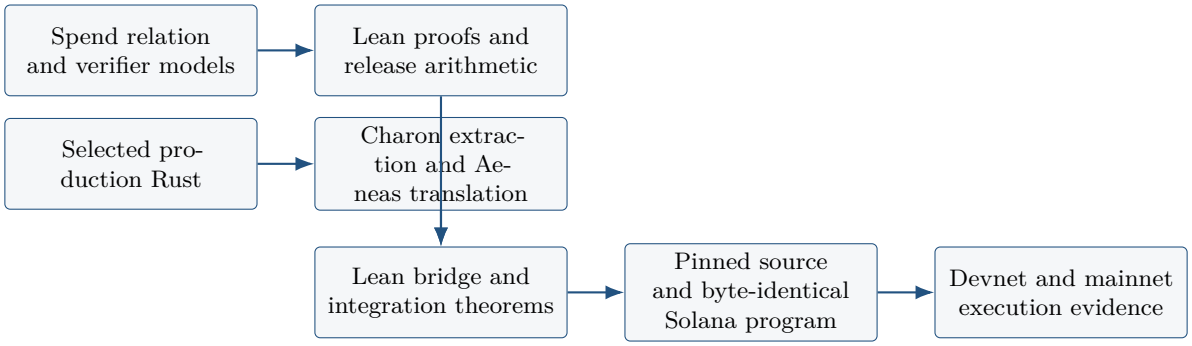

Figure 1: The Aspis evidence chain. The arrows do not all have the same status: some are Lean theorems, some are pinned-tool transformations, and the final arrow is an observed chain execution. The paper states the trusted boundary of each link.

Succinct pairing-based arguments make this practical in many deployed systems, but they usually introduce either a structured reference string or elliptic-curve assumptions. Transparent, hash-based proof systems remove the ceremony and offer a plausible route to post-quantum security, at the cost of substantially larger proofs and more expensive verification [1–4].

That trade-off is especially severe on Solana. A transaction has a strict compute budget, a small packet, bounded stack and heap behavior, and a public account-access discipline. A transparent proof that is routine on a server may be too large to transport in one packet, too expensive to verify, or unsafe to couple to state mutation. Prior work showed that a compact Winterfell STARK verifier can execute on Solana L1 within the compute limit [5]. Aspis asks a more application-specific question:

> Can a transparent proof certify a meaningful private-spend relation, and can the verifier apply the corresponding pool and double-spend state changes atomically, within one Solana transaction?

The answer demonstrated here is narrow but affirmative. Aspis verifies one private input note and one private output note against a sequential Merkle pool. The witness contains the input note data, owner secret, Merkle authentication path, output randomness, and integer values. The public statement contains the pool root and sequence, a one-time spent marker, the output commitment, the asset identifier, the public fee, and release-binding data. The relation checks ownership, membership, note and nullifier derivation, asset equality, bounded integer conservation, and a same-path root replacement. The final on-chain instruction reads a previously uploaded and sealed proof, reconstructs the public statement from instruction and account state, verifies the argument, checks the canonical spent-marker address, and applies the state transition only after all proof and account checks have succeeded.

The engineering result alone does not establish that the implemented verifier matches its mathematical description. Aspis therefore treats deployment as the last link in an evidence chain. The project has two connected formal layers. First, a Lean 4 development checks substantial parts of the mathematical construction and its release-specific arithmetic. Second, Charon extracts selected production Rust, Aeneas translates it into Lean, and bridge theorems connect the generated definitions to the maintained models. The compiled Solana program is then reproduced byte-for-byte from a pinned source tree and toolchain before runtime evidence is collected. Figure 1 summarizes this organization.

Two release lines. Aspis contains two related but distinct public artefacts. The q18/g37, Tag-65 release supplies the detailed finite soundness and hiding calculation and a finalized mainnet execution. Its proof is 65,407 bytes and the final transaction consumed 1,344,003 compute units. The V5, Tag-67 release is the current source-linked implementation: its 1,258,496-byte program is reproduced byte-for-byte from the recorded source, its selected production Rust is connected to Lean models through the Charon/Aeneas proof layer, and its full transition finalized on devnet at 1,335,952 compute units. A V5 mainnet signature was supplied while this manuscript was being prepared. The exact landed V5 metadata is intentionally kept separate until it is imported into an immutable release record. The later V5 formal work must not be read retroactively as a proof about the earlier Tag-65 binary.

Claims and non-claims. The base security statement is argument soundness: except with the stated error, an accepted proof implies the existence of a witness satisfying the spend relation. A theft-resistance corollary additionally assumes a cited round-by-round extraction or simulation-extractability premise. The paper keeps these claims separate. Likewise, "formalized" does not mean that every cryptographic primitive, every Rust function, the compiler, or the Solana runtime has been proved correct. Lean checks the specified theorem set; the Rust-to-Lean result covers selected release paths and retains one concrete hash-call equality. Build and runtime evidence constrain the remaining implementation links but do not turn them into theorems.

1.1 Contributions

The paper makes the following contributions.

C1. A transparent private-spend relation sized for Solana. We specify a one-input, one-output spend relation with note ownership, Merkle membership, a public one-time spent marker, asset preservation, integer value conservation, a public fee, and a same-path pool-root update. The relation uses Poseidon2 over the Mersenne-31 field for algebraic commitments and a circle-domain, WHIR-style commitment layer for proof verification.

C2. An atomic Solana state transition. We implement a proof-account lifecycle that transports a proof larger than the transaction packet, seals it by digest, and verifies it in a final instruction. The final instruction validates the pool, proof, payer, System Program, and canonical spent-marker accounts; reconstructs the statement; verifies the proof; rechecks mutable state; and only then advances the pool and records the spent marker. Failure before completion leaves persistent state unchanged.

C3. A two-layer formal development. The maintained Lean project proves substantial portions of the spend relation, finite release-security arithmetic, circle-group and hiding lemmas, fixed Poseidon2 executions, V5 Component-A/B/C models, retry control, and work authentication. Charon and Aeneas then translate selected production Rust into Lean, where bridge theorems establish agreement with the maintained models for the stated release path.

C4. An explicit implementation boundary. The final V5 integration theorem covers the selected Component-A schedule, Component B, Component C public output, Tag-67 byte decoding, and six ordered work checks. Its remaining code-to-model premise is a single, concrete equality asserting that the production transcript call hashes the state together with the domain byte and little-endian nonce specified by the Lean interface.

C5. Reproducible binary and execution evidence. The V5 Solana program is rebuilt byte-for-byte from a pinned public source commit and pinned build archives. The repository records devnet execution, current-runtime replay over all selectors and accepted marker pre-states, compute ceilings, rejected replay evidence, and offline release checks. The q18 release additionally records a finalized mainnet transaction and immutable proof/program identities.

C6. A conservative security and limitations record. The q18 parameter calculation is stated only in the proven Johnson/mutual-correlated-agreement regime; it does not rely on the now-disproved up-to-capacity conjectures [6, 7]. We distinguish per-random-oracle-query soundness from cumulative advantage, argument soundness from knowledge, proof-view hiding from transaction-graph privacy, and the demonstrated transition from a deployable private-payment system.

1.2 Organization

Section 2 reviews transparent commitments, private payments, Solana verification, and Rust verification. Section 3 defines the system and threat model. Section 4 specifies the spend relation and atomic state transition. Section 5 describes the q18 construction and the V5 implementation line at the level needed to understand the release. Section 6 states the security claims and assumptions. Section 7 presents the Lean and Charon/Aeneas evidence. Section 8 describes the Solana program and build binding. Section 9 reports execution and compute results. Section 10 records limitations, and Section 11 gives the reproduction procedure.

2 Related Work

Aspis sits at the intersection of transparent arguments, private ledger transitions, constrained on-chain verification, and source-linked formal verification. We organize related work by the role it plays rather than

by chronology.

2.1 Transparent proximity tests and polynomial commitments

FRI introduced a practically efficient interactive oracle proof of proximity for Reed–Solomon codes and became a central component of transparent arguments [1, 2]. DEEP-FRI improved soundness by extending the evaluation domain and constraining a prover outside the original codeword domain [3]. More recent work reduces verifier queries or broadens the queries supported by the commitment layer. STIR recursively improves code rate and obtains lower concrete query complexity [8]; WHIR supports constrained Reed–Solomon proximity testing and multilinear evaluation claims with very fast verification [4]. Brakedown, Spartan, and Orion explore different transparent or plausibly post-quantum routes to general-purpose arguments, with emphasis on prover time, field generality, or proof composition [9–11].

Aspis does not instantiate unmodified WHIR. Its q18 release uses a custom circle-domain, WHIR-style multilinear commitment layer adapted to a strict on-chain verifier budget. The implementation batches multilinear claims, performs four arity-four folds, uses few expensive queries, and compensates with explicit proof-of-work. Consequently, the paper maps each release claim to the exact code and finite parameter calculation rather than inheriting a headline security level from a named protocol.

Proximity-gap assumptions. The distinction between proven and conjectured distance regimes is now essential. Crites and Stewart disproved several up-to-capacity conjectures used in analyses of FRI-family systems, including the mutual-correlated-agreement conjecture stated by WHIR [6]. The 2026 systematization by Skatharoudis emphasizes the need to separate proven Johnson-bound guarantees from conjectural extensions [7]. Aspis's q18 release calculation is deliberately confined to its recorded, proven Johnson/MCA regime. No claim in this paper assumes mutual correlated agreement up to information-theoretic capacity.

2.2 Circle domains and Mersenne-31 arithmetic

Reed–Solomon codes over the circle group make efficient FFT-like evaluation available over fields whose multiplicative group does not have sufficient two-adicity [12]. Circle STARKs apply this observation to the Mersenne prime $p = 2^{31} - 1$, combining inexpensive word-size arithmetic with a large power-of-two subgroup on the circle curve [13]. Aspis adopts the same broad arithmetic setting: the relation and commitment layer use Mersenne-31 arithmetic, while extension fields provide sampling space and folding structure. The choice is motivated not only by prover speed but by the cost model of Solana's 32-bit SBF target.

2.3 Fiat–Shamir and concrete security

The Fiat–Shamir transform removes verifier interaction by deriving challenges from a transcript hash [14]. For multi-round proximity protocols, this step requires careful treatment: round-by-round soundness does not automatically imply the same non-interactive bound. Block et al. give a formal analysis of Fiat–Shamir security for FRI and related protocols [15], and Block and Tiwari evaluate concrete non-interactive FRI parameter sets, showing that provable and conjectured estimates can differ materially [16]. Aspis therefore publishes an event-by-event finite calculation, an explicit random-oracle query normalization, and a conservative whole-proof factor. The calculation is machine-checked in Lean for the release parameters, while its cryptographic formulae remain cited inputs.

2.4 Private ledger transitions

Zerocash formalized decentralized anonymous payments in which note origin, destination, and amount are hidden while a public ledger prevents double spending [17]. Modern shielded systems typically use note commitments, Merkle membership proofs, and nullifiers or spent markers. Aspis uses the same high-level ledger vocabulary but demonstrates a much narrower object. It has one input note, one output note, a public fee, a same-path replacement, and no deposit or append protocol. The main contribution is not a new full private currency; it is the execution and evidence chain for a transparent proof of one atomic spend transition under Solana's constraints.

2.5 On-chain transparent verification on Solana

Yano measured a fully on-chain pipeline that verifies a Winterfell STARK and a post-quantum signature on Solana, including proof upload, stack and allocator engineering, and use of Solana's SHA-256 syscall [5]. That work supplied the direct empirical starting point for Aspis: it showed that a transparent verifier could fit under the 1.4M compute-unit limit. Aspis differs in statement and integration. It proves a private-spend relation, verifies a much larger custom proof, and couples acceptance to a pool-root update and one-time spent marker in the same final transaction. The q18 release also operates on mainnet rather than only devnet. The comparison is therefore complementary: Yano studies a general feasibility and cost baseline; Aspis studies a specific private-state transition and its formal and release evidence.

2.6 Formal verification of Rust

Lean 4 is both a theorem prover and a functional programming language with a small trusted kernel [18]. Aeneas verifies Rust by translating a large safe subset into a pure functional representation, allowing proof engineers to reason about functional behavior rather than reconstructing low-level memory semantics [19]. Charon provides a stable analysis representation for Rust and serves as the extraction front end used by Aeneas and other tools [20].

Aspis uses these tools in a release-specific way. Selected production prover and verifier paths are extracted by Charon and translated by Aeneas. The generated Lean is not treated as self-explanatory: handwritten bridge theorems relate it to the independently maintained Aspis models. A final theorem packages the selected Component-A schedule, Components B and C, the public output, and Tag-67 work verification. This is more precise than saying that "the Rust implementation is formally verified," because large parts of the program, compiler, cryptographic primitives, and runtime remain outside that theorem.

2.7 Positioning

The closest technical influences provide individual parts of the Aspis design: FRI/STIR/WHIR supply proximity-testing techniques; circle-domain work supplies the arithmetic setting; Zerocash supplies the ledger pattern; Yano supplies the Solana verifier baseline; and Aeneas/Charon supply the implementation-translation pipeline. The claimed contribution is their release-bound composition: a stated private-spend relation, a substantial Lean development, selected Rust-to-model proofs, a byte-reproducible Solana binary, and observed chain execution. We make no broad priority claim over every prior private or transparent system on Solana. Any narrower novelty claim should be read against the dated prior-art record distributed with the repository.

3 System and Threat Model

3.1 Participants and ledger objects

Aspis has three logical participants.

- The *prover* holds the private input-note data, owner secret, Merkle path, output-note randomness, and the local state needed to generate and grind a proof.
- The *Solana program* receives public accounts and instruction bytes, reconstructs the statement, verifies the proof, and conditionally updates on-chain state.
- Any *observer* may inspect the program, proof account, transaction, account states, logs, fees, timing, and all other public chain data.

The persistent pool account stores a domain and program binding, a sequence number, and a Merkle root. Each accepted spend creates or finalizes a canonical program-derived spent-marker account associated with the public nullifier. A temporary program-owned proof account stores the uploaded proof bytes and lifecycle metadata. The payer and Solana System Program are supplied to create or normalize the spent marker and, for the q18 release, refund the proof account.

3.2 Adversarial capabilities

The adversary may generate arbitrary instruction bytes and proof bytes; choose malformed, aliased, prefunded, or incorrectly owned accounts; invoke the program at stale pool states; attempt to replay a proof; front-run a pending spend; and make adaptive random-oracle queries within the stated model. The adversary sees the complete public transcript and on-chain execution view. It may control the prover host except where fresh operating-system entropy is explicitly assumed.

The runtime is assumed to enforce Solana's account locking, ownership checks, transaction rollback, System Program semantics, and compute accounting. The security model does not attempt to hide the fee payer, timing, network origin, transaction graph, account-access pattern, or physical side channels. These leakages are outside the proof-view simulator.

3.3 Trusted and proved layers

Aspis separates four kinds of evidence.

1. **Mathematical theorems.** Lean checks statements about the maintained relation and verifier models, finite arithmetic, and selected hiding and group lemmas.
2. **Implementation correspondence.** Lean checks bridge theorems over definitions generated from selected Rust by Charon and Aeneas.
3. **Build identity.** A pinned toolchain reproduces the released V5 SBF bytes from a pinned source tree. This is a reproducibility record, not a compiler-correctness theorem.
4. **Runtime evidence.** Devnet, local-validator, Agave replay, and mainnet records show what the compiled program did on specified runtime versions. They do not prove behavior on every future runtime.

Cryptographic security of SHA-256 and Poseidon2, the applicability of cited proximity and Fiat–Shamir results, the remaining Rust hash-call equation, compiler correctness, and runtime correctness remain assumptions. Section 6 states them explicitly.

3.4 Goals

For a public statement x and witness w, the proof system aims to provide:

Completeness.
An honest prover with $(x, w) \in \mathcal{R}_{\text{spend}}$ produces a proof accepted by the verifier, subject to the release's bounded schedule-search policy.

Argument soundness.
An efficient adversary should not produce an accepted proof for a public statement with no satisfying witness, except with the stated error and random-oracle query dependence.

Conditional proof-view hiding.
For the declared proof and execution view, the distribution generated from either of two admissible witnesses should be computationally indistinguishable under the stated programmable-random-oracle, commitment, entropy, and masking assumptions.

Atomic ledger safety.
The pool update and spent-marker transition occur together only after proof, statement, account, and freshness checks succeed; a failed instruction leaves persistent state unchanged.

Replay prevention.
Once the canonical spent marker exists in an accepted state, the same public nullifier cannot be accepted again.

Release traceability.
The public source, formal artefacts, compiled program, proof and statement, runtime measurements, and chain evidence are bound by versioned manifests and digests.

3.5 Non-goals

The current release does not provide a complete anonymous-payment system. In particular, it does not provide a wallet, note discovery, deposits, append-only membership, multiple inputs or outputs, change, aggregation, concurrent spends within one pool, a growing anonymity set, transaction-graph privacy, or audited production security. The proof is uploaded through many preparatory transactions before the final state-changing transaction. These limitations are design facts, not merely missing user interfaces.

4 Spend Relation and Atomic Transition

This section gives a compact, implementation-oriented specification. The repository contains the exact byte encodings and Lean definitions; the purpose here is to expose the security-relevant structure without reproducing every field element.

4.1 Notes, commitments, and spent markers

Let $p = 2^{31} - 1$ and let $\mathbb{F}_p$ denote the Mersenne-31 field. A note contains an asset identifier a, an integer value v, an owner-derived key k, and randomness r. Its commitment is

$$c = \mathsf{Com}(a, v, k, r),$$

where Com is a domain-separated Poseidon2-based encoding over $\mathbb{F}_p$. The owner secret s determines the authorization key $k = \mathsf{H}_{\text{owner}}(s)$. A public one-time spent marker (a nullifier in standard terminology) is derived as

$$\nu = \mathsf{Null}(s, c, \mathsf{domain}),$$

again with a distinct domain-separated Poseidon2 call. The exact release encoding additionally binds the program and operator-selected domain values used by the pool.

The pool root commits to note leaves in a binary tree of depth 20. The demonstrated transition proves membership of one input leaf and replaces that leaf at the same path with one output commitment. Same-path replacement avoids proving an append protocol, but it also means that the release does not create a growing anonymity set.

4.2 Public statement and witness

We write the public statement as

$$x = (D, P, A, \mathsf{seq}, R_{\text{old}}, \nu, c_{\text{out}}, a, f, R_{\text{new}}, b),$$

where D is the release domain, P the program identity, A the pool identity, seq the expected pool sequence, R_{old} and R_{new} the old and new roots, ν the public spent marker, c_{out} the output commitment, a the asset identifier, f the public fee, and b denotes additional transcript and statement-binding values fixed by the release format.

The witness is

$$w = (s, a_{\text{in}}, v_{\text{in}}, r_{\text{in}}, v_{\text{out}}, r_{\text{out}}, \pi_{\text{Merkle}}, j, \omega),$$

where s is the owner secret, the two r values are note randomness, π_{Merkle} is the authentication path, j is the leaf index, and ω denotes proof-system masking randomness and auxiliary values.

Definition 4.1 (Spend relation)**.** $(x, w) \in \mathcal{R}_{\text{spend}}$ *when the following clauses hold.*

(i) the input authorization key is derived from s and the input note commitment is reconstructed correctly;
(ii) the reconstructed input commitment is a leaf under R_{old} at path j;
(iii) ν equals the release's domain-separated spent-marker derivation from the owner secret and input note;
(iv) c_{out} reconstructs from $(a, v_{\text{out}}, r_{\text{out}})$ and the output owner data fixed by the release statement;
(v) the input and output assets both equal the public asset a;
(vi) $v_{\text{in}}, v_{\text{out}}$, and f are in their declared integer ranges and

$$v_{\text{in}} = v_{\text{out}} + f$$

as an integer equality, not merely modulo p;
(vii) replacing the input leaf at path j with c_{out} yields R_{new};
(viii) all public binding, packing, and transcript clauses of the release hold.

The range argument decomposes 30-bit values using a 10-bit lookup table. The Lean relation proves that the range clauses prevent wraparound, so satisfaction of the field residuals implies the intended integer conservation law.

4.3 On-chain statement reconstruction

A proof-carried public statement is not trusted as an independent source of ledger facts. The final instruction reconstructs the public tuple from the instruction bytes and supplied accounts. In particular, the program reads the pool identity, sequence, and old root from the pool account; derives or checks the canonical spent-marker address from ν; binds the runtime program and release domain; and requires the claimed new root and output data to match the instruction encoding. The proof verifies only against this reconstructed tuple.

This design prevents a prover from presenting a proof about one state while mutating another. It also makes account validation part of the statement boundary: a cryptographically valid proof is insufficient if the accounts are stale, aliased, incorrectly owned, noncanonical, or incompatible with the instruction.

4.4 Atomic transition

At a high level, the V5 final instruction follows the sequence below.

T1. Parse an exact-length Tag-67 instruction and reject unknown tags, malformed fields, and trailing bytes.
T2. Validate the number, ownership, mutability, signer status, and distinctness of all accounts.
T3. Validate the proof account's lifecycle state, declared length, and digest; obtain the sealed proof bytes.
T4. Check the pool's domain, program binding, sequence, and root against the instruction.
T5. Derive and validate the canonical spent-marker program-derived address, including the required release bump.
T6. Reconstruct the public proof statement from instruction and account state.
T7. Verify the transparent argument and all ordered work checks.
T8. Re-read or recheck mutable state used before verification.
T9. Create, normalize, or validate the spent-marker account in one of the accepted pre-states.
T10. Advance the pool root and sequence; perform the release-specific proof-account disposition.

No persistent write occurs before proof acceptance and the final state checks. Solana's transaction rollback supplies all-or-nothing semantics if a later system call or account write fails. The verifier nevertheless treats rollback as a last line of defense: its explicit ordering is designed so that cryptographic failure and most structural failure occur before any mutation attempt.

Proposition 4.2 (Replay rejection, relative to runtime semantics)**.** *Assume the canonical spent-marker derivation is collision resistant for the release domain and Solana executes account ownership and transaction rollback as specified. After an accepted transition for ν, any later transaction using the same ν is rejected by the spent-marker state checks, unless the marker account or program semantics have been subverted.*

The proposition is a system statement rather than a pure cryptographic theorem: it depends jointly on the nullifier binding, canonical address derivation, account validation, persistent account state, and Solana runtime.

5 Transparent Argument Construction

The q18/g37 release is a transparent, WHIR-style polynomial argument over a circle evaluation domain. Its transcript, oracle layout, masking system, and relation checks are specific to Aspis; it is not an invocation of an unmodified WHIR parameter set. V5 retains the same broad proof-account and atomic-transition architecture but reorganizes the verifier into Components A, B, and C and uses its own release format. This section first describes the q18 construction, for which the complete parameter and security ledger is public, and then summarizes the V5 decomposition used by the source-linked formal proof.

5.1 Fields and hashes

The base field is

$$\mathbb{F} = \mathbb{F}_p, \qquad p = 2^{31} - 1.$$

The implementation uses the tower

$$\mathbb{F}_1 = \mathbb{F}[i]/(i^2 + 1), \qquad \mathbb{K} = \mathbb{F}_1[u]/(u^2 - (2 + i)),$$

so $|\mathbb{K}| = p^4$. The code calls these representations M31, CM31, and QM31. Canonical field encodings are little-endian and reject integers at least p.

The arithmetic relation uses a fixed width-16 Poseidon2 permutation over $\mathbb{F}$ [21]. Domain-separated calls implement owner derivation, note commitments, nullifiers, and two-to-one Merkle compression. SHA-256 is used for the Fiat–Shamir transcript because Solana exposes a native SHA-256 syscall. The resulting division of labor is intentional: Poseidon2 is inexpensive inside the algebraic relation, whereas SHA-256 is inexpensive inside the on-chain transcript verifier.

5.2 Trace and code parameters

The q18 trace has $2^{10} = 1024$ message rows and is encoded to 2^{19} circle symbols, an inverse rate of 512. The layer-zero codeword is grouped into $N = 2^{17}$ arity-four query fibers. The verifier samples $q = 18$ distinct fibers per branch, with three branches sharing the same transcript prefix. Four arity-four folds reduce the committed word to a terminal polynomial with four coefficients in $\mathbb{K}$.

Table 1: Frozen q18/g37 proof parameters.

Quantity	Value
Trace/message rows	2^{10}
Circle evaluation symbols	2^{19}
Inverse rate	512
Layer-zero fibers	2^{17}
Queries per branch	18, without replacement
Query branches	3
Attempt cap	17
Batch work	37 bits
Fold work	34, 33, 30, 25 bits
Final work	32 bits
Folds	four arity-four folds
Range lookup	exact 10-bit table

The arithmetic trace contains deterministic Poseidon2 blocks for owner derivation, input and output notes, the two Merkle paths, and the nullifier. Tagged copy constraints force the input and output path computations to share the same path index and siblings. Lookup and reconstruction constraints establish the 30-bit value ranges, after which the balance residual expresses an integer equality rather than an equality modulo p.

5.3 Committed lanes and batching

The layer-zero message contains 26 base-field codewords and three extension-field codewords. Sixteen base-field lanes carry the randomized semantic trace and ten carry mask-only directions. The extension lanes are denoted H, G, D. Two Merkle commitments authenticate the base-field and extension-field leaves. After sampling a nonzero challenge $\gamma \in \mathbb{K}^*$, the verifier combines the 29 lanes as

$$R_\gamma = \sum_{j=0}^{25} \gamma^j C_j + \gamma^{26} H + \gamma^{27} G + \gamma^{28} D.$$

The masking construction includes root-neutral directions: changes in semantic or mask lanes are paired with compensating changes in G or D so that the committed batched word is unchanged while the opened view gains entropy. The release-specific *Good* predicate checks the necessary source guards and rank conditions for the selected transcript schedule.

Each query opens one arity-four fiber from the two layer-zero commitments, checks the batched value, verifies the first fold, and then checks the authenticated line layers and terminal polynomial. Each fold round also carries out-of-domain values and a relation sumcheck.

5.4 Fiat–Shamir schedule and work

The transcript is typed and domain-separated. In simplified form, it:

1. absorbs the release header, field basis, statement digest, and masking context;
2. absorbs the two layer-zero roots and samples relation and batching challenges;
3. verifies a masked sumcheck and derives its terminal point;
4. absorbs the relation evaluations at the required statement points;
5. checks a 37-bit batch-work nonce before sampling γ;
6. for each of four folds, absorbs the round data, checks the round work nonce, and samples the fold challenge;
7. absorbs the terminal polynomial and checks the 32-bit final-work nonce;
8. forks into three branches, each of which absorbs one branch byte and samples 18 distinct query fibers.

The work nonces are not a trusted setup and do not hide any trapdoor. They make a forged transcript more expensive to search. Under the random-oracle model, each accepted work nonce contributes its leading-zero requirement to the finite event calculation. This design trades prover time for verifier operations, which is favorable under a hard on-chain compute ceiling.

The prover is allowed up to 17 complete schedule attempts. It fails closed if no schedule passes the release's Good predicates. Lean checks the exact retry bound and the authentication/order of the six work values in V5. Availability of a good schedule is a liveness property; soundness does not condition on the prover successfully finding one.

5.5 V5 component decomposition

The V5 verifier is organized into three proof-facing components and a final work verifier.

Component A.
Matrix execution and the selected GoodA release schedule. The maintained Lean model includes rank and terminal-applicability results. The production-Rust theorem is specialized to the selected release schedule.

Component B.
A ten-round sampler/evaluator path, triangular hiding structure, commitment/transcript binding, and the C2 layout. The generated Rust definitions are connected to the corresponding Lean terminal model.

Component C.
Sampler and pivot encoding, the QM31 tower and codec, residual projection, four runtime folds, finalization, packing, and the deployed public rows.

Work verifier.
Tag-67 carries six work values: batch, four fold, and final. Exact-length decoding, little-endian reads, projection to the leading-zero predicate, and ordered checking are linked to Lean, with one remaining premise for the underlying transcript hash call.

This decomposition is partly an implementation strategy and partly a proof strategy. It permits selected production paths to be translated and related to smaller maintained models. It must not be confused with a theorem about every byte of the proof format or every verifier function; the exact theorem scope is given in section 7.

5.6 Why these parameters fit Solana

The release optimizes for the number of on-chain field and hash operations rather than for proof size or prover time. Three choices dominate:

- The Mersenne-31 field makes reduction inexpensive on the SBF target and supports the circle domain.
- A very low code rate increases per-query detection power, allowing only 18 queries per branch.
- Grinding supplies additional soundness without adding Merkle openings or fold checks to the verifier.

The result is operationally awkward but computationally feasible: a 65,407-byte q18 proof and many preparatory upload transactions, followed by a final verification and state update under 1.4M compute units. V5 increases the program size and changes the proof-facing decomposition while preserving a small margin under the same transaction ceiling.

6 Security Claims and Assumptions

This section states what the release claims and, equally importantly, what it does not claim. The q18/g37 release has the complete public finite-event ledger. V5 has its own implemented endpoint and formal component models, but this draft does not silently transfer every numerical q18 theorem to V5. The final V5 paper revision should cite the exact V5 release certificate once the mainnet record is frozen.

6.1 Argument soundness

Let Q_H be the number of relevant adversarial SHA-256 random-oracle queries in the Fiat–Shamir reduction. The q18 release combines the following failure sources: proximity-test error in a proven Johnson/MCA regime, batching and sampling events, finite-field challenge collisions, relation/commitment opening error, the three branch union, and the multi-round Fiat–Shamir loss. The event ledger is computed from the frozen manifest and checked in Lean.

Theorem 6.1 (q18 work-normalized argument soundness, informal)**.** *Under the published Johnson/MCA and PCS/BCS results mapped by the release, collision resistance and random-oracle modeling of the transcript hash, and the stated field and binding assumptions, an adversary that causes the q18 verifier to accept a public statement x for which no w satisfies $(x, w) \in \mathcal{R}_{\text{spend}}$ has advantage at most*

$$Q_H \cdot 2^{-100.16}$$

under the release's conservative whole-proof normalization, plus separately listed primitive and implementation error terms.

The displayed quantity is a per-random-oracle-query floor, not a claim of a fixed 100-bit bound against an unbounded adversary. Cumulative advantage grows linearly with Q_H in this accounting and becomes vacuous at sufficiently large budgets. The repository publishes a table of concrete budgets rather than reporting only the rounded "100-bit" label.

The theorem asserts witness existence. It does not, by itself, assert that an accepted prover *knows* the owner secret in the sense needed to rule out all theft strategies. This distinction matters for a payment application.

6.2 Theft resistance and knowledge

A theft-resistance statement requires connecting an accepting proof to an extractable witness and then using note/nullifier binding to show that the witness authorizes the public spent marker. Aspis formalizes the connective implication but treats the round-by-round extractor and simulation-extractability premise as an external cryptographic input.

Theorem 6.2 (Theft resistance, conditional)**.** *Assume the argument satisfies the cited round-by-round extraction and simulation-extractability property; Poseidon2 note, owner, and nullifier calls satisfy their stated binding and preimage properties; and the program enforces the canonical spent marker. Then an efficient adversary cannot spend an honestly formed note without producing an authorizing witness, except with the sum of the extraction, argument, primitive, and runtime error terms.*

If the extractor premise fails, definition 6.1 may still imply that some satisfying witness exists, but the stronger theft-resistance interpretation no longer follows. This is why the public documentation avoids using "proof of knowledge" as a synonym for the base soundness theorem.

6.3 Proof-view hiding

The q18 release includes a conditional computational hiding argument for the declared proof view. The simulator programs the SHA-256 random oracle, uses the release masking directions, and relies on rank conditions checked for the public schedule. Under the stated masking, entropy, commitment, and programmable-random-oracle assumptions, the published calculation gives approximately a 2^{-104} real-versus-simulator floor and a 2^{-103} pairwise witness-indistinguishability floor.

These numbers cover the modeled proof-and-execution view. They do not hide the fee payer, timing, transaction graph, upload pattern, network metadata, pool choice, or physical side channels. Nor do

they create a meaningful anonymity set in the demonstrated pool: because there is no append or deposit path, the public experiment had one replaceable leaf. The formal hiding result is therefore best read as a statement about the proof representation, not as a claim of deployed payment anonymity.

6.4 Proximity regime after the 2025 conjecture results

The q18 calculation is bound to a proven Johnson/MCA regime. This qualification is not cosmetic. Crites and Stewart showed that several stronger Reed–Solomon proximity-gap conjectures fail up to capacity, including the mutual-correlated-agreement conjecture used in WHIR's conjectural analysis [6]. The current literature therefore distinguishes:

- guarantees proved up to Johnson-type or list-decoding regimes;
- revised conjectures up to list-decoding capacity; and
- disproved claims up to information-theoretic capacity.

Aspis uses only the first category for its published q18 floor. The custom use of WHIR-style folding does not authorize importing WHIR's fastest conjectural parameter estimates. The paper's theorem map must be rechecked whenever the commitment layer or parameters change.

6.5 Assumptions and trusted boundaries

Table 2 summarizes the principal boundaries. A complete release ledger names the evidence constraining each assumption and the consequence if it fails.

Table 2: Principal security and implementation assumptions.

Boundary	Used for	Consequence if false
Published Johnson/MCA and PCS/BCS results apply to the frozen construction and parameters	Argument soundness, opening security, work-normalized endpoint	The mapped soundness theorem does not follow
Fiat–Shamir through SHA-256 is modeled as a programmable random oracle	Non-interactive soundness and simulation	The Fiat–Shamir reduction and hiding simulator do not follow
SHA-256 has the required collision, preimage, and random-oracle properties	Transcript binding, Merkle authentication in the proof layer, work, simulation	Transcript or work binding may fail
Poseidon2 over M31 has the required security, and the deployed calls satisfy the relation interface	Notes, owner derivation, nullifiers, relation binding	Commitment, authorization, or spent-marker binding may fail
Round-by-round extractor and simulation-extractability premise	Theft-resistance corollary	Soundness may remain, but the knowledge/theft interpretation fails
Fresh independent operating-system entropy	Masking, salts, schedule attempts	Hiding or retry-distribution claims may fail
Exact Tag-67 transcript hash-call equation	Last Rust-to-Lean link for work verification	The decoded work theorem may not describe the runtime digest
Lean/mathlib, Charon, Aeneas, Rust, LLVM, and pinned build tools operate correctly	Proof checking, extraction, translation, compilation, binary identity	A theorem or binary may not represent the intended source
Solana account locking, rollback, System Program, SHA syscall, and compute accounting behave as recorded	Atomicity, replay prevention, runtime cost	State or cost claims may not hold on the affected runtime

6.6 The remaining Tag-67 hash-call equation

Pinned Aeneas cannot translate the production transcript's function-pointer field. The final Tag-67 implementation theorem therefore assumes exactly

$$\forall\, state, nonce, \quad \mathsf{actualTranscriptGrindingDigest}(state, nonce) = \mathsf{rustHash}\big(state,\ 3 \parallel \mathsf{nonceLE64}(nonce)\big).$$

The byte 3 is the release's grinding domain. Given this equation and successful generated byte guards, the theorem concludes the little-endian projection, leading-zero predicate, and order of the batch, four fold, and final checks. The equation is intentionally concrete: it is not an unrestricted assumption that "the Rust matches the Lean."

6.7 Atomic state safety

The cryptographic theorem and the state-transition theorem have different premises. Let S be a valid pre-state and let I be a final instruction whose accounts satisfy the structural checks. Relative to Solana's rollback and System Program semantics, the program has the following property:

Proposition 6.3 (Fail-closed transition)**.** *If proof verification, account validation, canonical spent-marker validation, or the final mutable-state recheck fails, the transaction does not change the pool root, pool sequence, or accepted spent-marker state. If the transaction succeeds, it changes the pool from the public old root/sequence to the public new root/next sequence and records the public spent marker.*

This proposition does not claim that a malicious validator implementation preserves Solana semantics. It is a program property evaluated under the recorded runtime model and reinforced by hostile-path tests and chain evidence.

7 Formal Development

Aspis uses two connected formal proof layers. The first formalizes the mathematical construction in Lean 4. The second connects selected production Rust to those models using Charon, Aeneas, and additional Lean theorems. The distinction is central: a theorem about an independently written model does not by itself establish that the deployed program implements that model.

7.1 Lean formalization of the construction

The maintained `AspisFormal/` project covers both the q18 release and the V5 verifier models. Its audited integration theorems depend on Lean and mathlib's standard logical base {`propext`, `Classical.choice`, `Quot.sound`}. The maintained project reports no `sorry`, custom axiom, `native_decide`, or compiled-evaluation shortcut. Fixed Poseidon2 known-answer executions use kernel `decide` on pinned transitions.

q18 relation and security arithmetic. The q18 development includes:

- integer value conservation after the 30-bit range constraints;
- derivation of the range, balance, and asset clauses from the arithmetic residuals;
- the maintained spend relation from Poseidon2 and Merkle clauses, relative to the named all-input `Poseidon2Faithful` interface;
- the manifest-bound Johnson/MCA regime and agreement cap;
- the complete finite-event soundness calculation and work-normalized endpoint, relative to the cited BCS/Fiat–Shamir formulae;
- circle-generator order, same-x criteria, and query-fiber root distinctness;
- distribution-level masking and concrete circle-matrix hiding for the stated model;
- pinned Poseidon2 permutation, node, owner, note, and nullifier known-answer bindings;
- the implication from the extractor premise and nullifier binding to theft resistance.

The formalization therefore checks the finite arithmetic and internal logical composition of the release while preserving the distinction between proved lemmas and imported cryptographic results.

V5 mathematical models. The V5 Lean layer includes:

- Component-A rank, schedule, and terminal-applicability theorems;
- Component-B triangular hiding, spend-difference coverage, sumcheck commitment, and transcript binding;
- Component-C sampling, pivot encoding, QM31 tower/codec, residual projection, concrete fold linearity, conditional hiding, and deployment composition;
- the Good-gate verifier relation and functional batching;
- the exact 17-attempt retry controller and nonce/work authentication;
- the implemented work-normalized endpoint under the cited cryptographic formula.

Some V5 hiding statements are proved relative to named entropy, transcript, commitment, serialization, and hash interfaces. This is normal proof engineering: the theorem records the precise interface required rather than turning an unproved primitive claim into a hidden axiom.

7.2 From production Rust to Lean

The source-linked pipeline has four stages.

1. Charon extracts selected V5 Rust paths from the production prover and verifier into a stable low-level representation [20].
2. Aeneas translates the extracted definitions into pure Lean definitions [19].
3. Handwritten Lean bridge proofs relate the generated definitions to the maintained Aspis models.
4. A final integration theorem packages the selected Component-A, Component-B, Component-C, public-output, and Tag-67 work-verifier results under their stated hypotheses.

Lean checks the generated definitions and bridge proofs together. The retained translations were produced with Aeneas revision

`b59d5188c082f704a418c7cb4e52ad69328002d1`

and Charon revision

`cb50ff16b9f1066b8a97dc06da704de2da2fa41c`.

The release keeps normalized generated Lean, proof source, theorem maps, and replay scripts on the main branch; bulk extraction history is retained in an archive tag.

7.3 Current V5 theorem map

Table 3 gives the principal release theorems. The long identifiers are included because they are stable review entry points, not because they improve exposition.

Table 3: Principal V5 production-Rust correspondence results.

Path	Result	Principal theorem
Component A	Extracted matrix execution agrees with the maintained GoodA model for the selected release schedule	`FormalClosureStream1.component_a_actual_matches_maintained`
Component B	Generated sampler, evaluator, and C2 layout agree with the maintained ten-round model	`FormalClosureStream1.component_b_actual_matches_maintained`
Component C	Four runtime rounds, finalization, packer, and public rows agree with the deployed evaluator model	`generated_public_run_output_matches_deployed`
Tag-67 work wire	Generated guards and little-endian reads produce the decoded Lean work view; the proof-facing chain preserves all six checks	`AspisTag67WorkVerifierClosure.tag67AcceptedWireAndVerifierClosure`
Final V5 composition	Packages the selected A result, B, C public output, and Tag-67 work verification under canonical-input and execution hypotheses	`FormalClosureStream1.current_source_combined_capstone`

The Component-A theorem is release-schedule specific. The production verifier recomputes GoodA and GoodB for every accepted selection, but a universal source theorem connecting every possible extracted schedule to the general model remains open. The Component-B, Component-C, public-output, Tag-67 decoding, and ordered-work theorems have the broader scopes recorded in their theorem ledgers.

7.4 The transcript function-pointer boundary

Aeneas cannot translate the production `Transcript` function-pointer field used by the SHA-256 call. The six-step control-flow proof therefore uses a small proof-facing Rust helper that receives the booleans computed by the production digest checks. Separate theorems prove the digest predicate and all byte projections, leaving only the equality in section 6.

This boundary is smaller than a generic source-to-model premise. Once the equality is supplied, the theorem derives:

- exact acceptance of the Tag-67 magic and length guards;
- little-endian decoding of the six work values;
- the domain byte and nonce encoding used in the grinding digest;
- the leading-zero predicate; and
- the order batch → fold 0 → fold 1 → fold 2 → fold 3 → final.

A future proof could remove the premise by translating or separately verifying the concrete SHA wrapper. Until then, the equation is exposed in the theorem statement and release assumptions.

7.5 What the formal development does not prove

The combined theorem is not an end-to-end verification of the complete Rust program. In particular, it does not establish:

- one universal theorem for every verifier function and every proof byte;
- comprehensive joint serialization faithfulness for the whole cryptographic view;
- a universal all-input Rust equality for Poseidon2;
- first-principles proofs of the imported PCS, proximity, Fiat–Shamir, or primitive-security results;
- correctness of Charon, Aeneas, Lean, mathlib, the Rust compiler, LLVM, the SBF toolchain, or Solana;
- the account-validation and mutation path as a single machine-checked Solana semantics theorem.

These omissions do not negate the formal results; they define their exact scope. The release combines them with parameter-binding CI, known-answer tests, hostile account-path tests, source-to-binary reproduction, and runtime evidence. Each form of evidence addresses a different failure mode.

7.6 Replay and auditability

The maintained mathematical project builds with:

```
cd AspisFormal
lake exe cache get
lake build
```

The retained Rust-to-model packages replay under pinned Lean 4.32 through two release entry points:

```
aeneas-verif/component-c-runtime-downstream/
  released-trace-families-current-20260722/replay-lean432.sh

aeneas-verif/current-source-abc-capstone-20260722/
  replay-lean432.sh
```

Authenticated dependency caches and explicit tool-location variables permit a reviewer to reuse or rebuild the pinned environment without committing generated object files. The replay scripts are part of the release evidence, not informal instructions copied from a development machine.

8 Solana Implementation

8.1 Program organization

The implementation separates shared cryptographic logic, statement construction, proof generation, Solana dispatch, and release tooling. The host and program share field and transcript code where possible. The q18 verifier is exposed through the statement crate; the V5 production path enters through the Solana dispatcher's Tag-67 branch, which supplies the strict V5 verifier to an atomic verify-and-apply wrapper.

The default production grammar is append-only. Diagnostic and historical tags are excluded from the accepted route, and unknown tags fail before account access. For V5, Tag 67 is the only full verify-and-apply instruction in its release family; Tag 66 is not accepted by the production dispatcher.

8.2 Proof-account lifecycle

A q18 proof is 65,407 bytes, far above Solana's 1,232-byte transaction packet limit. Aspis therefore transports proofs through a program-owned account:

1. create the proof account with a declared capacity and authority;
2. append fixed-offset chunks, rejecting overlaps and out-of-range writes;
3. finalize the account only when the complete byte image has the expected digest;
4. execute the final verification and state transition against the sealed bytes;
5. close or retain the proof account according to the release path.

The q18 release uses 69 upload chunks, so the final verification is preceded by 71 preparatory transactions: create, 69 appends, and finalize. Its final path refunds the proof account. V5 retains the sealed proof during the state transition and closes it through a separate, evidence-gated cleanup command after finalization. In both cases, the phrase "no trusted setup" refers to cryptographic parameters; these upload transactions are data transport, not a setup ceremony.

The sealed account is not accepted merely because it has the right length. The verifier checks owner, discriminator/lifecycle state, finalized authority sentinel, declared proof length, and digest. This prevents post-finalization modification and binds the final transaction to a byte-exact proof.

8.3 Tag-67 instruction and account checks

The Tag-67 public wire has a fixed grammar. The parser rejects the wrong tag, wrong exact length, invalid canonical encodings, and trailing data. The public tuple is reconstructed rather than accepting a second, proof-carried copy of live pool facts.

The V5 atomic path receives the proof account, pool state, canonical spent-marker account, payer, and System Program. Before verification it checks, among other conditions:

- pairwise distinctness of proof, pool, spent marker, and payer;
- program ownership and discriminators of the pool and accepted marker forms;
- signer and writable requirements;
- pool domain, sequence, and root;
- canonical spent-marker seeds and the release-required bump 255;
- proof-account lifecycle and byte identity;
- absence of sequence overflow;
- accepted marker pre-state: absent, program-owned valid form, or the explicitly measured prefunded System-owned form.

The test suite constructs hostile role reuse: the proof account is reused as the pool or marker, the pool is reused as the marker, the payer is reused as a writable state account, accounts have foreign ownership or wrong discriminators, markers are pre-seeded in foreign forms, and mutable state changes between the initial snapshot and final commit. Each test asserts both the exact rejection and unchanged persistent state.

8.4 Verify before writing

The high-cost proof verification executes after structural validation but before any persistent state write. After verification, the program reborrows and rechecks mutable pool and marker state. This second check protects against implementation errors or test harnesses that mutate state during a long verification path. Only then does the program invoke any required System Program operations for the marker and write the new pool root and sequence.

The accepted prefunded-marker path is the longest measured state transition because it may require transfer, allocate, and assign CPIs before the program writes the marker. This path determines the V5 release-policy ceiling. The executor simulates the exact signed transaction bytes and refuses submission above 1,356,912 compute units; the transaction declares the same compute limit.

8.5 Transcript and hash implementation

The host and SBF code implement a byte-exact SHA-256 transcript with typed domain labels, canonical little-endian encodings, challenge sampling, sampling without replacement, and work predicates. The Solana path calls the native SHA-256 syscall to avoid a prohibitively expensive software hash. Poseidon2 remains in field arithmetic for note and relation operations.

The source-linked formal proof covers the Tag-67 byte guards, reads, and ordered work logic, but the concrete SHA function-pointer call is the explicit premise described in sections 6 and 7. Independent transcript order tests and known-answer fixtures constrain this boundary.

8.6 Source-to-binary identity

The V5 release binary has length 1,258,496 bytes and SHA-256

`4cf3c1d5ddd47efa68875c0070247e007083c5c9bb2d5988db0d644a609edf40`.

The candidate tag points to commit

`d98a2e69618dc95b95c1996506ed8c05904a56a1`,

while the exact build-source commit is

`06788d44d30ea8cbd391899dddaf6f0acc6e4a3f`.

A clean clone of the latter reproduced the binary byte-for-byte using pinned Agave/platform-tools archives whose digests are recorded by the release. The release distinguishes the source tree that built the binary from the later metadata tree that contains the complete evidence.

This record reduces a common ambiguity in deployed-program papers: the public source is not merely asserted to be “the code”; a specific tree and environment reproduce the exact bytes measured and deployed. Compiler and toolchain correctness remain trusted, but accidental source/binary drift becomes directly testable.

8.7 One-shot mainnet executor

The V5 mainnet path generates a fresh statement and proof bound to the canonical program ID, fresh pool, sequence zero, anchor, nullifier, and mainnet domain. A read-only readiness command checks the binary and provenance, runtime identity, funding, signers, ProgramData allocation, statement/proof identity, canonical marker bump, and exact signed-transaction simulation.

The execution command persists each Rust-signed transaction before submission, records the CLI deployment command, submits the same Tag-67 bytes that were simulated, and refetches the finalized transaction. An interrupted run stops for manual reconciliation; it does not automatically resume or resubmit an irreversible mainnet step. Post-execution cleanup commands require the finalized Tag-67 evidence before closing the proof account, ProgramData, or dedicated payer.

9 Evaluation

The evaluation has two purposes. First, it measures whether the complete verifier and state transition fit within Solana's transaction limits. Second, it records enough identity information to connect the measured execution to the source, proof, statement, and formal release described in the preceding sections. We report the earlier q18/g37 mainnet release because its immutable evidence bundle is complete, then the V5 devnet and current-runtime measurements, and finally the V5 mainnet signature supplied for this manuscript. We do not infer missing landed fields from a web explorer.

9.1 Methodology

All compute figures refer to the final transaction that verifies a sealed proof and applies the state transition. Deployment, pool creation, proof-account creation, chunk uploads, proof finalization, and later cleanup are reported separately. The relevant hard limit is 1,400,000 compute units (CU). Solana transactions are also limited to 1,232 serialized bytes, which is why the proof is stored in a program-owned account before the final instruction [22].

The release methodology distinguishes four measurements:

1. *host verification*, which checks the proof outside SBF and catches format or fixture errors;
2. *local-runtime replay*, which executes accepted and rejected account topologies under a recorded Agave version and feature set;
3. *exact-wire simulation*, which signs the transaction, serializes the final bytes, and simulates those exact bytes against the target cluster; and
4. *landed execution*, which refetches the finalized transaction and records the observed slot, logs, compute consumption, account identities, and post-state.

A simulation result is not reported as a mainnet execution. Likewise, a signature produced locally is not treated as evidence of inclusion. The release record must show a successful finalized transaction and bind it to the same instruction bytes and program identity that were simulated.

9.2 Finalized q18/g37 mainnet release

The q18/g37 release executed on Solana mainnet on 16 July 2026. Its final transaction is

`3G1voggszvDMGi5PbGM1kuEMYKvh2TNMbH6hHHwndUdRQJNT7ehRFpQpksxLnx5tp2xkS5jGi359rVXk42sRPFcv`.

The transaction finalized at slot 433,219,840 and consumed 1,344,003 CU. The released proof is 65,407 bytes. The exact release identities are summarized in table 4.

Table 4: Immutable q18/g37 mainnet release.

Item	Value
Release tag	`aspis-spend-q18-g37-mainnet-v1`
Program ID	`GQPNqfYF17Nj2dGsf6Q2AtiouyM67YxFZPh9LxBk2Ye3`
Compiled program	924,344 bytes
Program SHA-256	`e289faf85fe4773880794d5d4356461bb9cb94077b68711f99348efe19707d7e`
Proof	65,407 bytes
Proof SHA-256	`32eb419e0c5c3ef4fa2db0d32579e88f1207547d8fb010279efeb6c05981b529`
Final transaction	`3G1voggszvDMGi5PbGM1kuEMYKvh2TNMbH6hHHwndUdRQJNT7ehRFpQpksxLnx5tp2xkS5jGi359rVXk42sRPFcv`
Finalized slot	433,219,840
Compute consumption	1,344,003 CU
Headroom below 1.4M	55,997 CU
Pool	`3ZRYarQZcWJQgzNwLo1Eo7PDmier3rbW41L8ccAddCvb`
Spent-marker account	`CFJ5fYPSi4Qn6okBCZS3tXFMw7Lsz4Jy9vEAAGU5kfSU`
Lifecycle after demonstration	Program and temporary accounts closed as recorded by the bundle

The final verifier transaction was preceded by proof-account creation, 69 chunk uploads, and proof finalization. These 71 preparatory transactions transport and seal proof data; they are unrelated to the absence of a cryptographic trusted setup. The final instruction verified the sealed proof, advanced the pool, created the spent marker, and refunded the proof account atomically.

The q18 release gate measured 1,344,057 CU as its maximum accepted path, leaving 55,943 CU below the limit. The landed transaction used 54 CU less. This small difference is consistent with the landed account topology being slightly cheaper than the conservative accepted-path gate; it is not treated as evidence that all future runtime versions preserve the same margin.

9.3 V5 binary, devnet execution, and current-runtime replay

V5 is a distinct release line rather than a relabeling of q18. Its candidate binary is 1,258,496 bytes with SHA-256

`4cf3c1d5ddd47efa68875c0070247e007083c5c9bb2d5988db0d644a609edf40`.

A clean clone of source commit

`06788d44d30ea8cbd391899dddaf6f0acc6e4a3f`

reproduced these bytes exactly. The candidate metadata is carried by commit

`d98a2e69618dc95b95c1996506ed8c05904a56a1`.

This separation permits later evidence files to describe a fixed binary without pretending that the metadata commit itself produced it.

The full V5 Tag-67 state transition finalized on devnet in transaction

`38mNKeMmRf9Ttqmde8jZqCMq8F1piHhCEqGYVYrSHEggTu21C93wWAEhoDkZ2qJHeTX7j3qCmibNNmZ6awWtEuLC`

at slot 478,299,357, consuming 1,335,952 CU. The proof was 75,838 bytes, with SHA-256

`cc53b00b7c82b008ba554c6e3860431e0f3796e60ad50eac2126e431f14c9fdd`.

The 768-byte statement record had SHA-256

`5c8c609f8a53f0c8928d9353b6220119cb089d95efe7ff8bb1feac0e1db04e4e`.

The release also retains selector-specific accepted proofs between 72,958 and 75,358 bytes; proof length therefore depends on the selected V5 trace family rather than being a single protocol constant.

Table 5: V5 evidence available before importing the final mainnet record.

Measurement	Value
Declared mainnet program ID	`7Q2nGsPg8rbjdxKHK4jxTgEWLTyd9o1X4KMSjCieRmue`
Compiled program	1,258,496 bytes
Program SHA-256	`4cf3c1d5ddd47efa68875c0070247e007083c5c9bb2d5988db0d644a609edf40`
Clean source rebuild	Byte-identical
Devnet final transaction	`38mNKeMmRf9Ttqmde8jZqCMq8F1piHhCEqGYVYrSHEggTu21C93wWAEhoDkZ2qJHeTX7j3qCmibNNmZ6awWtEuLC`
Devnet proof	75,838 bytes
Devnet transaction CU	1,335,952
Current replay runtime	Agave 4.1.0; feature set 3,345,198,602
Observed replay slot	434,766,704
Selector 0 / 1 / 2 prefunded-marker CU	1,334,528 / 1,337,192 / 1,329,776
Exact signed-wire policy ceiling	1,356,912 CU
Policy headroom	43,088 CU
Required spent-marker PDA bump	255

The V5 release-policy ceiling is not simply the largest of the three selector figures in table 5. It also includes the complete accepted-marker topology family and the exact signed-wire runner policy. The longest accepted path is the prefunded System-owned marker case, which may require transfer, allocate, and assign operations before the program writes the marker. The runner requires the canonical PDA bump 255, signs and serializes the transaction, simulates those exact bytes, and refuses submission above 1,356,912 CU. It sets the transaction compute limit to the same value.

9.4 V5 mainnet transaction

The V5 mainnet transaction supplied for this manuscript is

`EJviPgF12i9iK2CveVaQSMeFQqDMFPQ1iPRUYEwNQE3zGquTUZNJXPZEENorcQtsnQj1orFmH1TPsgdbR3vJ2fE`.

The signature is included so that the manuscript and release work can proceed without ambiguity about the intended chain event. At the cut-off for Draft 0.1, however, the repository's authoritative release-facts record still described V5 as not deployed and did not contain a final V5 mainnet object. The public explorer page exposed to the drafting environment did not provide machine-readable landed metadata. We therefore record only the signature as observed input and leave all derived fields open.

Table 6: V5 mainnet execution record for Draft 0.1. "Open" fields must be imported from the immutable release evidence before submission.

Field	Draft 0.1 value
Final transaction signature	`EJviPgF12i9iK2CveVaQSMeFQqDMFPQ1iPRUYEwNQE3zGquTUZNJXPZEENorcQtsnQj1orFmH1TPsgdbR3vJ2fE`
Cluster	mainnet-beta, as stated by the author; must be confirmed by genesis hash
Finalized status and slot	**Open – import RPC finalization record**
Block time	**Open – import RPC record**
Landed compute consumption	**Open – import transaction metadata**
Serialized transaction and message digests	**Open – compare with persisted signed bytes**
Instruction tag and exact wire digest	**Open – decode and bind to the Tag-67 release wire**
Deployed program ID	Expected `7Q2nGsPg8rbjdxKHK4jxTgEWLTyd9o1X4KMSjCieRmue`; **confirm**
ProgramData address and deployed bytes	**Open – import deployment evidence**
Deployed program SHA-256	Expected `4cf3c1d5ddd47efa68875c0070247e007083c5c9bb2d5988db0d644a609edf40`; **confirm**
Proof account, proof bytes, and proof SHA-256	**Open – import final proof identity**
Statement bytes and statement SHA-256	**Open – import final statement identity**
Pool account and pre/post sequence/-root	**Open – import finalized readbacks**
Spent-marker account and pre/post owner/data	**Open – import finalized readbacks**
Rejected replay transaction or simulation	**Open – import replay evidence**
Deployment, setup, and cleanup signatures	**Open – import lifecycle record**
Fees, rent held, and rent refunded	**Open – reconcile lamports**

The final manuscript should replace table 6 with a compact immutable table and move the full record to an appendix or release artifact. The minimum acceptance checks are:

1. the transaction is finalized on mainnet-beta and succeeded;
2. the invoked upgradeable program resolves to ProgramData whose extracted bytes hash to the released V5 SBF;
3. the persisted locally signed transaction is byte-identical to the landed transaction or has an explicitly explained transport transformation;

4. the instruction decodes as the exact Tag-67 wire whose simulation was below the policy ceiling;
5. the sealed proof and statement match their published hashes;
6. the pool sequence and root changed to the proof's public post-state;
7. the canonical spent-marker account changed from an accepted pre-state to the terminal spent state;
8. a replay using the same spent marker is rejected without changing state; and
9. cleanup occurs only after the final transaction and readbacks are preserved.

Until those checks are incorporated, the signature establishes a review target but not the complete evidence chain claimed for the final release.

9.5 Cost distribution and bottlenecks

For both release lines, the dominant cost is cryptographic verification rather than the final pool write. SHA-256 transcript operations use Solana's native syscall; field arithmetic, Merkle opening checks, fold evaluation, and relation-specific work dominate the remaining verifier. The V5 binary is roughly 36% larger than the q18 binary, but its devnet final transaction is slightly cheaper than the q18 mainnet transaction. Program size and transaction compute are therefore not monotone: V5 carries additional code and release paths while reorganizing the hot verifier logic.

The proof transport cost is primarily operational rather than computational. Chunk append instructions are cheap, but their number produces latency, fees, and failure-handling burden. Proof generation is also dominated by release grinding: on the author's older MacBook, the complete local search is reported to take approximately eight minutes. This value is hardware- and entropy-dependent and should be replaced in the final evaluation by a pinned machine specification, repeated trials, median, range, and a breakdown between trace generation, commitment, folding, and grinding.

9.6 What the evaluation demonstrates

The completed q18 record demonstrates that a 65-kilobyte transparent private-spend proof can be checked and coupled to a Solana state transition below the mainnet transaction limit. The V5 devnet and replay records demonstrate that the source-linked V5 verifier, larger proof family, and more complete release discipline also fit with approximately 43,000 CU of policy headroom under the recorded runtime. The supplied V5 signature indicates that the final mainnet step has been attempted or completed, but the paper will claim the V5 landed result only after the immutable release record closes the fields above.

The evaluation does not show competitive payment throughput, a useful anonymity set, or safety under future runtime repricing. Those limitations are addressed next.

10 Limitations and Open Problems

Aspis is a feasibility result and an assurance artifact, not a deployable private-payment system. This section states the limitations that materially constrain interpretation of the result.

10.1 Application scope and anonymity

The demonstrated relation has one input, one output, one public fee, and a same-path replacement in a single sequential pool. There is no deposit or append path. The public experiments therefore do not create a growing anonymity set; the demonstrated pool has one replaceable position. A proof-view hiding theorem can show that selected witness data is hidden by the proof representation while the system still provides little transaction-level anonymity. These are different properties.

A practical pool would require at least note insertion, multiple concurrently spendable notes, a commitment-tree update policy, wallet state recovery, multiple inputs and outputs, change handling, and a scalable spent-marker structure. Each addition changes the statement, trace, proof size, formal relation, account topology, and soundness calculation. The current numbers should not be extrapolated to such a system without a new release analysis.

10.2 Sequential execution

Every spend is bound to the current pool sequence and root, and the pool account is writable during the final transaction. Two spends against one pool therefore cannot finalize concurrently. In addition, the next proof must be generated against the state produced by the previous spend. The approximate eight-minute local proving and grinding time, rather than Solana's block rate, bounds per-pool throughput.

Independent pools permit horizontal scaling but partition the anonymity set and liquidity. Removing this bottleneck likely requires a queue, batched state transition, append-only commitment structure, or proof that tolerates a controlled set of intervening updates. Those designs introduce new consistency and front-running questions.

10.3 Proof transport and storage

A 65–76 kilobyte proof cannot fit in a Solana transaction. Uploading a sealed proof account costs many preparatory transactions and temporarily locks rent. A production protocol would need reliable resumable upload, fee estimation, garbage collection, denial-of-service protection, and a policy for stale proofs. The V5 executor deliberately stops for manual reconciliation after an interruption because blind resubmission of irreversible mainnet steps is unsafe. That behavior is suitable for a one-shot research release, not for a wallet.

Each accepted spend also leaves a canonical spent-marker account. This makes replay state simple to audit but grows storage linearly. A practical design would likely replace one account per spend with a batched tree, accumulator, or queue, which would change both compute and proof statements.

10.4 Runtime margin

The q18 transaction leaves about 56,000 CU below the 1.4M limit; the V5 release policy leaves 43,088 CU. These margins are small relative to the whole verifier. Solana may reprice syscalls, alter loader behavior, activate features, or change account-operation costs. The release therefore binds compute claims to an observed Agave version and feature set and requires current-cluster exact-wire simulation. A future runtime can make the same binary exceed its limit without any source change.

This sensitivity also limits composability. The final Aspis transaction has little room for additional application logic, token movement, wallet checks, or cross-program calls. Integrating the verifier into a broader protocol may require parameter changes or splitting responsibilities across transactions, which would weaken the present all-in-one statement.

10.5 Cryptographic assumptions

The q18 argument-soundness floor is conditional on the cited proximity, polynomial-commitment, batching, and Fiat–Shamir results applying to the exact custom construction. The 2025 counterexamples to up-to-capacity proximity conjectures show why this mapping cannot be treated as routine. Aspis confines its published q18 calculation to a proven Johnson/MCA regime, but independent cryptographic review of the custom circle-domain WHIR-style layer remains valuable.

Theft resistance requires more than argument soundness. It additionally assumes the stated round-by-round extraction and simulation-extractability property. The Lean development proves consequences of that premise; it does not prove the premise from first principles. Poseidon2 is also used in a custom M31 setting for note, owner, nullifier, and tree operations. Known-answer tests and formal parameter bindings establish implementation identity, not primitive security.

The hiding result is conditional and view-specific. It uses a programmable-random-oracle model, masking and entropy assumptions, and declared public leakage. It does not establish network-level anonymity, unlinkability of fee payers, timing privacy, or side-channel resistance.

10.6 Formal-verification boundary

The formal development is substantial but deliberately scoped. Lean checks mathematical models and finite calculations; selected Rust is translated and related to those models. It does not prove every line of the prover, verifier, transaction executor, account mutation code, compiler, or runtime. The final Tag-67

work theorem retains one explicit equation connecting the production transcript hash call to the modeled SHA invocation.

Charon, Aeneas, Lean, mathlib, the Rust compiler, LLVM, the SBF toolchain, and the Solana runtime remain in the trusted computing base. Source-to-binary reproduction detects drift but does not prove compiler correctness. Hostile account tests provide evidence about state safety but are not a mechanized semantics proof of Solana execution.

A valuable next formal step is to translate or verify the concrete transcript SHA wrapper and eliminate the remaining hash-call equation. A larger step is an end-to-end theorem over a model of the Tag-67 account transition, including parser, account validation, System Program interactions, and writes. Such a theorem would still depend on a correspondence between the model and Solana's actual runtime.

10.7 Audit and testing

The repository has internal red-team review, property and hostile-path tests, known-answer fixtures, formal proofs, independent arithmetic checkers, source/binary reproduction, and chain evidence. It has not received an independent cryptographic audit or Solana program audit, and it does not yet publish a coverage-guided fuzzing campaign. No real value should be placed under the current protocol on the basis of this paper alone.

Priority external-review targets are the custom PCS and its concrete soundness mapping, the Poseidon2-M31 instantiation, the transcript and serialization boundary, Rust paths outside the Aeneas coverage, account aliasing and marker normalization, and release recovery after interrupted transaction sequences.

10.8 Novelty scope

We do not claim that Aspis is the first privacy system, shielded transaction, transparent proof, or on-chain verifier associated with Solana. Prior systems cover nearby categories, and terminology such as "shielded spend" is not uniform. The narrower contribution is the documented composition of a private-spend relation, direct transparent verification in a Solana program, an all-or-nothing pool and spent-marker update, substantial Lean formalization, selected Rust-to-model correspondence, byte-reproducible deployment, and mainnet evidence. The repository distributes a dated prior-art record and invites corrections.

11 Reproducibility and Artifact Structure

The artifact is organized so that a reviewer can check different links independently. A successful Lean build does not substitute for a binary rebuild; a matching binary does not substitute for chain evidence; and an explorer link does not substitute for an offline release record.

11.1 Repository layout

The principal directories are:

`AspisFormal/`
: Maintained Lean models and proofs for the relation, security arithmetic, circle and hiding lemmas, Poseidon2 known-answer bindings, and V5 verifier models.

`aeneas-verif/`
: Retained Charon/Aeneas-generated Lean, handwritten bridge proofs, theorem maps, and pinned replay scripts for selected production Rust.

`crates/aspis-core/`
: Shared host and verifier arithmetic, transcript, and proof logic.

`crates/aspis-statement/`
: Private-spend relation, statement encoding, and q18 verifier integration.

`crates/aspis-prover/`
: Proof generation, grinding, release fixtures, and independent finite-event calculators.

`programs/aspis-verifier/`
: Solana dispatcher, proof-account lifecycle, V5 verifier, account validation, and atomic state transition.

release/
: Immutable or candidate release packages, manifests, checksums, proofs, binaries, formal entry points, and chain evidence.

results/
: Runtime replay and measurement records whose identity is linked from release manifests.

tools/ and xtask/
: Release consistency checks, source/binary reproduction, transaction execution, and evidence collection.

The exact directory names are implementation identifiers. The final public release should provide a short top-level path that runs the checks below without requiring a reviewer to reconstruct the research history.

11.2 Checking the mathematical proofs

The maintained Lean development is built with:

```
cd AspisFormal
lake exe cache get
lake build
```

The proof-status ledger names each principal theorem, its hypotheses, and whether the result is proved internally or depends on a named cryptographic interface. CI should build the project with the pinned Lean toolchain, scan the maintained proof surface for forbidden placeholders, and preserve the theorem-dependency report used by the release.

11.3 Replaying the Rust-to-Lean proofs

The retained V5 integration packages replay through:

```
aeneas-verif/component-c-runtime-downstream/
  released-trace-families-current-20260722/replay-lean432.sh

aeneas-verif/current-source-abc-capstone-20260722/
  replay-lean432.sh
```

A reviewer should check both the generated definitions and the bridge proofs. The release records the Aeneas and Charon revisions, normalized extracted output, authenticated dependency caches, and principal theorem names. Re-extracting from Rust is a stronger check than replaying retained Lean alone; the final artifact should make both paths explicit and report any normalization applied between extraction and proof replay.

11.4 Rebuilding the Solana program

The V5 source-to-program check uses the recorded build-source commit, pinned Agave archive, platform-tools archive, Cargo lock file, feature selection, architecture, and offline build command. It then compares the resulting file byte-for-byte and recomputes SHA-256. The required outcome is:

```
bytes  = 1258496
sha256 = 4cf3c1d5ddd47efa68875c0070247e007083c5c9bb2d5988db0d644a609edf40
```

A fail-closed provenance workflow should treat an unavailable SBF toolchain, skipped build, hash mismatch, or missing paper build as a failure rather than a warning. The source commit that produced the binary and the later release-metadata commit must be recorded separately.

11.5 Checking the release packages

The q18 and V5 candidate packages provide offline verifiers:

```
./release/aspis-spend-q18-g37-mainnet-v1/verify.sh
./release/aspis-v5-tag67-frozen-candidate-v1/verify.sh
python3 tools/check_release_facts.py
```

The scripts check the manifest and all released bytes against `SHA256SUMS`, including proofs, statements, compiled programs, runtime records, and formal entry-point files. The facts checker also scans public documentation for stale identities or status claims. For the final V5 release, the verifier must additionally require non-null finalized deployment and execution records and must bind the exact mainnet signature from section 9.4.

11.6 Independent chain verification

A chain reviewer should not rely on a single explorer. Starting from the release manifest, the reviewer should use a mainnet RPC to:

1. fetch the transaction at finalized commitment and save the complete JSON response;
2. verify the cluster genesis hash;
3. resolve the invoked upgradeable program to ProgramData and reconstruct the deployed ELF/SBF bytes;
4. compare their hash with the release manifest;
5. decode the transaction message and instruction bytes;
6. compare the message and serialized-transaction digests with the locally persisted signed bytes;
7. fetch the pool, proof, and spent-marker accounts at the relevant slots where archival data permits;
8. verify the post-state and a rejected replay; and
9. preserve all RPC responses in the immutable release package with endpoint identity redacted only where credentials require it.

The final release should include a small provider-independent script using standard Solana JSON-RPC. Provider-specific examples, including Helius or other indexed services, may improve accessibility but must not become a correctness dependency.

11.7 Artifact claims matrix

Table 7: Which artifact checks support which claims.

Claim	Primary check	Important residual trust
Mathematical lemma holds	Lean kernel checks maintained theorem	Lean, mathlib, theorem hypotheses
Selected Rust agrees with model	Charon/Aeneas output plus Lean bridge theorem	Extraction/translation tools and uncovered Rust
Published source builds published program	Pinned clean build and byte comparison	Compiler and toolchain correctness
Program executed on target cluster	Finalized RPC transaction and ProgramData readback	Solana consensus/runtime and RPC honesty
State update was all-or-nothing	Program ordering, hostile tests, landed pre/post state	Runtime rollback and System Program semantics
Security number matches parameters	Independent calculator plus Lean finite proof	Imported cryptographic formulae and primitive assumptions

11.8 Submission artifact freeze

Before ePrint or conference submission, the following objects should be immutable and mutually cross-referenced:

- source and metadata commits;
- final release tag;
- compiled program and SHA-256;
- proof and statement bytes and hashes;
- Lean and Aeneas/Charon revisions and theorem entry points;
- runtime identity and exact-wire simulation;
- deployment, final transaction, replay, and cleanup evidence;
- paper PDF and SHA-256;
- bibliographic metadata and DOI, when assigned.

The living repository may continue after publication, but the paper must cite an immutable release.

This separation prevents later documentation or code changes from silently altering the object evaluated by reviewers.

12 Conclusion

Aspis investigates whether a transparent private-spend proof and its ledger effects can fit inside one Solana transaction. The q18/g37 release gives a complete mainnet record: a 65,407-byte proof was verified, the pool was advanced, and a one-time spent marker was recorded at a cost of 1,344,003 CU. V5 develops the same research direction with a larger source-linked assurance package, a byte-reproducible 1,258,496-byte Solana program, a finalized devnet transition at 1,335,952 CU, and a current-runtime policy ceiling of 1,356,912 CU. The final V5 mainnet signature is identified in this draft; the submission version will import its complete landed evidence from the immutable release record.

The main contribution is not the compute figure in isolation. Aspis connects several kinds of evidence that are often published separately: a precise private-spend relation; substantial Lean formalization of the construction and release arithmetic; Charon/Aeneas translation of selected production Rust; Lean proofs connecting that code to the maintained models; reproducible source-to-program identity; and observed chain execution. The boundaries between these layers are stated explicitly, including the remaining transcript-hash equation and the cryptographic assumptions used for soundness, hiding, and theft resistance.

The result remains deliberately narrow. It does not provide a wallet, deposit path, growing anonymity set, concurrent pool, or production audit. Its proof transport and grinding costs are impractical, and its compute margin is sensitive to runtime repricing. Nevertheless, it establishes a concrete point in the design space: transparent private-state verification with an all-or-nothing Solana update is possible within today's transaction limit, and the path from mathematical model to deployed bytes can be made substantially more reviewable than a conventional benchmark release.

A Release Identities

Table 8: Principal immutable and candidate identifiers used in Draft 0.1.

Object	Identifier
q18 release tag	`aspis-spend-q18-g37-mainnet-v1`
q18 tag commit	`cdb3d3408b89b5179353d01cffea2b0ef6484fb3`
q18 program ID	`GQPNqfYF17Nj2dGsf6Q2AtiouyM67YxFZPh9LxBk2Ye3`
q18 program SHA-256	`e289faf85fe4773880794d5d4356461bb9cb94077b68711f99348efe19707d7e`
q18 proof SHA-256	`32eb419e0c5c3ef4fa2db0d32579e88f1207547d8fb010279efeb6c05981b529`
q18 mainnet transaction	`3G1voggszvDMGi5PbGM1kuEMYKvh2TNMbH6hHHwndUdRQJNT7ehRFpQpksxLnx5tp2xkS5jGi359rVXk42sRPFcv`
V5 candidate tag	`aspis-v5-tag67-frozen-candidate-v1`
V5 candidate commit	`d98a2e69618dc95b95c1996506ed8c05904a56a1`
V5 candidate tree	`919f461131dbdceb8fa00783a16658fa10059ac6`
V5 build-source commit	`06788d44d30ea8cbd391899dddaf6f0acc6e4a3f`
V5 build-source tree	`9b6bdfddb3c213addc2bb705c8130cce4fb2c351`
V5 declared program ID	`7Q2nGsPg8rbjdxKHK4jxTgEWLTyd9o1X4KMSjCieRmue`
V5 program SHA-256	`4cf3c1d5ddd47efa68875c0070247e007083c5c9bb2d5988db0d644a609edf40`
V5 devnet proof SHA-256	`cc53b00b7c82b008ba554c6e3860431e0f3796e60ad50eac2126e431f14c9fdd`
V5 devnet statement SHA-256	`5c8c609f8a53f0c8928d9353b6220119cb089d95efe7ff8bb1feac0e1db04e4e`

Object	Identifier
V5 devnet transaction	`38mNKeMmRf9Ttqmde8jZqCMq8F1piHhCEqGYVYrSHEggTu21C93wWAEhoDkZ2qJHeTX7j3qCmibNNmZ6awWtEuLC`
V5 mainnet transaction supplied for draft	`EJviPgF12i9iK2CveVaQSMeFQqDMFPQ1iPRUYEwNQE3zGquTUZNJXPZEENorcQtsnQj1orFmH1TPsgdbR3vJ2fE`
Aeneas revision	`b59d5188c082f704a418c7cb4e52ad69328002d1`
Charon revision	`cb50ff16b9f1066b8a97dc06da704de2da2fa41c`

B Formal Review Entry Points

This appendix lists stable review entry points. The theorem statements and their hypotheses in the release tree are authoritative.

Table 9: Principal formal entry points.

Area	Review entry point
q18 value conservation	Lean theorem chain deriving integer balance from range and residual clauses in `AspisFormal/`
q18 relation	Maintained spend-relation theorem relative to `Poseidon2Faithful`
q18 security endpoint	Manifest-bound Johnson/MCA and finite-event theorem ending at the work-normalized q18 endpoint
Circle and hiding results	Generator order, same-x criteria, query-fiber distinctness, masking, and concrete circle-matrix theorems
V5 Component A	`FormalClosureStream1.component_a_actual_matches_maintained`
V5 Component B	`FormalClosureStream1.component_b_actual_matches_maintained`
V5 Component C output	`generated_public_run_output_matches_deployed`
Tag-67 work wire	`AspisTag67WorkVerifierClosure.tag67AcceptedWireAndVerifierClosure`
Final V5 composition	`FormalClosureStream1.current_source_combined_capstone`
Maintained Lean build	`cdAspisFormal&&lakeexecacheget&&lakebuild`
Component-C replay	`aeneas-verif/component-c-runtime-downstream/released-trace-families-current-20260722/replay-lean432.sh`
Final V5 replay	`aeneas-verif/current-source-abc-capstone-20260722/replay-lean432.sh`

The principal unresolved implementation equation is

$$\forall\, state, nonce, \quad \mathsf{actualTranscriptGrindingDigest}(state, nonce) = \mathsf{rustHash}\big(state, 3 \parallel \mathsf{nonceLE64}(nonce)\big).$$

Reviewers should read this equation together with the generated byte guards and the proof of the six ordered work checks. It is the last explicit source/model premise for that Tag-67 path, not a general assumption that all Rust code agrees with Lean.

C Submission-Critical Open Fields

Draft 0.1 deliberately leaves the following fields open. They are release-data tasks, not invitations to estimate values in prose.

1. Import the finalized V5 mainnet transaction response for `EJviPgF12i9iK2CveVaQSMeFQqDMFPQ1iPRUYEwNQE3zGquTUZNJXPZEENorcQtsnQj1orFmH1TPsgdbR3vJ2fE`, including slot, block time, status, fee, logs, and compute consumption.
2. Import the deployment transaction and resolve the program to ProgramData; reconstruct the deployed bytes and confirm the V5 program SHA-256.

3. Publish the exact mainnet proof and statement bytes, lengths, hashes, selector, and canonical public-input digest.
4. Compare the persisted signed message and serialized transaction with the landed transaction and publish both digests.
5. Record pool and spent-marker pre-state and finalized post-state, including sequence, roots, owner, discriminator, and lamports.
6. Publish a rejected replay of the same spent marker and show that persistent state remains unchanged.
7. Reconcile all deployment, preparation, proof-account, marker, cleanup, fee, and rent movements.
8. Replace the approximate proving-time statement with repeated measurements on a named machine and software build.
9. Freeze a final V5 release tag, paper PDF hash, citation metadata, and DOI.
10. Rerun the complete fail-closed provenance workflow from the final tag and archive its outputs.

No submission should claim a complete V5 mainnet evidence chain while any of Items 1–7 remain absent from the immutable release package.

References

[1] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. *Scalable, Transparent, and Post-Quantum Secure Computational Integrity*. Cryptology ePrint Archive, Paper 2018/046. 2018. URL: https://eprint.iacr.org/2018/046.

[2] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. "Fast Reed–Solomon Interactive Oracle Proofs of Proximity". In: *45th International Colloquium on Automata, Languages, and Programming*. 2018, 14:1–14:17. DOI: 10.4230/LIPIcs.ICALP.2018.14.

[3] E. Ben-Sasson, L. Goldberg, S. Kopparty, and S. Saraf. *DEEP-FRI: Sampling Outside the Box Improves Soundness*. Cryptology ePrint Archive, Paper 2019/336. 2019. URL: https://eprint.iacr.org/2019/336.

[4] G. Arnon, A. Chiesa, G. Fenzi, and E. Yogev. *WHIR: Reed–Solomon Proximity Testing with Super-Fast Verification*. Cryptology ePrint Archive, Paper 2024/1586. 2024. URL: https://eprint.iacr.org/2024/1586.

[5] J. Yano. *Full L1 On-Chain ZK-STARK+PQC Verification on Solana: A Measurement Study*. Cryptology ePrint Archive, Paper 2025/1741. 2025. URL: https://eprint.iacr.org/2025/1741.

[6] E. Crites and A. Stewart. *On Reed–Solomon Proximity Gaps Conjectures*. Cryptology ePrint Archive, Paper 2025/2046. 2025. URL: https://eprint.iacr.org/2025/2046.

[7] C. Skatharoudis. *SoK: Hash-Based Polynomial Commitments and Low-Degree Tests: From FRI to BaseFold, STIR, and WHIR*. Cryptology ePrint Archive, Paper 2026/1367. 2026. URL: https://eprint.iacr.org/2026/1367.

[8] G. Arnon, A. Chiesa, G. Fenzi, and E. Yogev. *STIR: Reed–Solomon Proximity Testing with Fewer Queries*. Cryptology ePrint Archive, Paper 2024/390. 2024. URL: https://eprint.iacr.org/2024/390.

[9] A. Golovnev, J. Lee, S. Setty, J. Thaler, and R. S. Wahby. *Brakedown: Linear-time and Field-agnostic SNARKs for R1CS*. Cryptology ePrint Archive, Paper 2021/1043. 2021. URL: https://eprint.iacr.org/2021/1043.

[10] S. Setty. "Spartan: Efficient and General-Purpose zkSNARKs Without Trusted Setup". In: *Advances in Cryptology - CRYPTO 2020*. 2020, pp. 704–737. DOI: 10.1007/978-3-030-56877-1_25.

[11] T. Xie, Y. Zhang, and D. Song. *Orion: Zero Knowledge Proof with Linear Prover Time*. Cryptology ePrint Archive, Paper 2022/1010. 2022. URL: https://eprint.iacr.org/2022/1010.

[12] U. Haboeck, D. Lubarov, and J. Nabaglo. *Reed–Solomon Codes over the Circle Group*. Cryptology ePrint Archive, Paper 2023/824. 2023. URL: https://eprint.iacr.org/2023/824.

[13] U. Haboeck, D. Levit, and S. Papini. *Circle STARKs*. Cryptology ePrint Archive, Paper 2024/278. 2024. URL: https://eprint.iacr.org/2024/278.

[14] A. Fiat and A. Shamir. "How to Prove Yourself: Practical Solutions to Identification and Signature Problems". In: *Advances in Cryptology - CRYPTO 1986*. 1987, pp. 186–194.

[15] A. R. Block, A. Garreta, J. Katz, J. Thaler, P. R. Tiwari, and M. Zajac. *Fiat–Shamir Security of FRI and Related SNARKs*. Cryptology ePrint Archive, Paper 2023/1071. 2023. URL: https://eprint.iacr.org/2023/1071.

[16] A. R. Block and P. R. Tiwari. *On the Concrete Security of Non-interactive FRI*. Cryptology ePrint Archive, Paper 2024/1161. 2024. URL: https://eprint.iacr.org/2024/1161.

[17] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza. *Zerocash: Decentralized Anonymous Payments from Bitcoin*. Cryptology ePrint Archive, Paper 2014/349. 2014. URL: https://eprint.iacr.org/2014/349.

[18] L. de Moura and S. Ullrich. "The Lean 4 Theorem Prover and Programming Language". In: *Automated Deduction - CADE 28*. 2021, pp. 625–635. DOI: 10.1007/978-3-030-79876-5_37.

[19] S. Ho and J. Protzenko. "Aeneas: Rust Verification by Functional Translation". In: *Proceedings of the ACM on Programming Languages* 6.ICFP (2022), pp. 711–741. DOI: 10.1145/3547647.

[20] S. Ho, G. Boisseau, L. Franceschino, Y. Prak, A. Fromherz, and J. Protzenko. *Charon: An Analysis Framework for Rust*. arXiv:2410.18042. 2024. URL: https://arxiv.org/abs/2410.18042.

[21] L. Grassi, D. Khovratovich, and M. Schofnegger. *Poseidon2: A Faster Version of the Poseidon Hash Function*. Cryptology ePrint Archive, Paper 2023/323. 2023. URL: https://eprint.iacr.org/2023/323.

[22] Solana Foundation. *Solana Transaction and Program Documentation*. 2026. URL: https://solana.com/docs.